# Unit 4: Sensors & Actuators with Arduino Uno

**Ultra-quick, complete exam revision | Read the red “Memory” lines before the exam.**

## 1. The complete IoT idea

**IoT system = Physical thing + Controller + Sensors + Actuators + Network.** A sensor takes input from the environment. Arduino is the controller/brain: it reads input, decides, and controls an actuator. An actuator produces a physical output.

**Flow:** Environment → Sensor → Arduino → Actuator → Action. Example: smoke → MQ-2 → Arduino → relay/fan/alarm ON.

**Memory: Sensor senses; Arduino thinks; actuator acts.**

| Classification | Types / meaning |
|---|---|
| By power | Active: requires power supply. Passive: does not require external power. |
| By output | Analog: continuously varying voltage/value. Digital: only HIGH/LOW, 1/0. |

## Useful Arduino functions

**pinMode(pin, INPUT/OUTPUT)**: sets pin type. **analogRead(A0)**: reads analog value. **digitalRead(pin)**: reads HIGH/LOW. **digitalWrite(pin, HIGH/LOW)**: controls output. **Serial.begin(9600)**: starts serial communication. **Serial.print/println**: displays data. **delay(ms)**: pauses program.

## 2. Gas and obstacle sensors

**MQ-02/MQ-05 gas sensor:** MQ-2 detects smoke, H2, LPG, CH4/methane and alcohol. MQ-5 detects natural gas and LPG; it is less smoke-sensitive. Pins: **VCC→5V, GND→GND, AO→analog output, DO→digital output.** Flammable gas reduces internal resistance, causing a changing/higher analog voltage at AO. Use **analogRead(A0)** to print gas level.

MQ series: MQ-3 alcohol/ethanol; MQ-4 methane/CNG; MQ-6 LPG/butane; MQ-7 CO; MQ-8 H2; MQ-9 CO/flammable gases; MQ-131 ozone; MQ-135 air quality; MQ-136 H2S; MQ-137 ammonia; MQ-138 benzene/toluene/alcohol/acetone/propane/formaldehyde/H2.

**Memory: Gas ↑ → resistance ↓ → analog voltage ↑.**

**Obstacle sensor:** A digital IR sensor used to detect objects. VCC→5V, GND→GND, OUT→Arduino digital input. Its IR emitter sends IR; receiver gets reflected IR. Detection range is set by its potentiometer. **Obstacle present → reflected IR → LOW/0. No obstacle → HIGH/1.**

**Memory: Obstacle → reflection → LOW.**

## 3. HC-SR04 Ultrasonic Distance Sensor

**Purpose:** measures distance only; it cannot identify the type of object. Pins: VCC→5V, GND→GND, **Trigger** sends pulse, **Echo** receives echo. Typical connection: Trigger pin 11, Echo pin 12.

| Feature | Value |
|---|---|
| Frequency | 40 kHz |
| Range | 2 cm to 400 cm |
| Sound speed | 343 m/s = 0.0343 cm/µs |
| Trigger action | HIGH for 10 µs |
| Transmission | 8 ultrasonic pulses |

**Working:** (1) Make Trigger HIGH for 10 µs. (2) Module sends 8 pulses. (3) Pulses hit object and return. (4) Echo remains HIGH for the round-trip travel time. (5) Arduino measures this duration. (6) Divide by 2 because sound travels to the object and back.

**Distance in cm = (duration ÷ 2) ÷ 29.1 = (duration ÷ 2) × 0.0343**

**delayMicroseconds(x)** pauses for x microseconds. **pulseIn(echoPin, HIGH)** measures duration of HIGH pulse. If no echo returns, Echo times out and goes LOW after about 38 ms.

**Memory: Trigger sends, Echo listens, time gives distance.**

## 4. LDR Light Sensor

**LDR = Light Dependent Resistor.** It measures light intensity and works in a voltage-divider circuit. Connect a 10 kΩ resistor from 5V to A0; connect LDR from A0 to GND. Arduino reads A0 using **analogRead(A0)**.

| Condition | LDR resistance | Voltage at A0 |
|---|---|---|
| Dark | High | Near 0 V |
| Bright light | Low | Near 5 V |

Example decision: value **<400** means “Darker”; value **>550** means “Too Bright”.

**Memory: Light ↑ → LDR resistance ↓.**

## 5. Heartbeat Sensor

**Purpose:** measures human heartbeat/heart rate in health-care and wearable systems. Pins: VCC→5V, GND→GND, A0→Arduino analog input. Place a finger on the sensor: during a heartbeat blood volume in capillary tissue increases, changing reflected/transmitted light. This creates an output pulse.

**Program idea:** read A0; detect a pulse above UpperThreshold (518); save first and second pulse times using **millis()**; calculate PulseInterval; when reading goes below LowerThreshold (490), allow the next pulse to be detected.

**BPM = (1 / PulseInterval in ms) × 60 × 1000 = 60,000 / PulseInterval**

**Memory: Blood change → light change → pulse → time gap → BPM.**

## 6. Actuators

**Actuator:** an output device that moves or controls a system after Arduino makes a decision from sensor data. Examples: LED, light, motor, relay, fan, pump and servo motor.

| Actuator type | Energy / principle |
|---|---|
| Hydraulic | Uses hydraulic liquid power for mechanical operation |
| Pneumatic | Uses compressed air or vacuum for linear/rotary motion |
| Electrical | Uses electrical energy |

**Memory: Sensors know; actuators do.**

## 7. Relay

**Relay:** electrically activated switch. A 5 V relay module lets a low-power Arduino pin control high-power loads such as AC bulbs, fans and motors. The module uses a transistor to operate its coil.

| Terminal | Meaning | |
|---|---|---|
| COM | Common terminal | |
| NC | Normally Closed terminal | |
| NO | Normally Open terminal | |
| Relay state | Coil current | COM connection |
| OFF | No current | COM connected to NC |
| ON | Current flows | COM connected to NO |

Connection: Arduino **5V→relay VIN/VCC, GND→GND, D6→Signal**. Code: set relay pin as OUTPUT; HIGH turns it ON and LOW turns it OFF (as shown in the slides).

**Memory: Relay OFF → NC; Relay ON → NO.**

**Applications:** home automation, switching AC loads/heavy loads, timed control, safety circuits, automobile indicators and motors.

**Safety:** 230 V AC mains is dangerous; use isolation and proper supervision.

## 8. Servo Motor

**Servo motor:** a linear/rotary actuator that moves to a commanded position, such as 0°, 90° or 180°. It is a **closed-loop system**: position feedback adjusts motor motion until the required final position is reached. Arduino's Servo library generates the required control signal.

| Method | Use |
|---|---|
| attach(pin) | Connect servo variable to Arduino pin |
| write(angle) | Move shaft to requested angle |
| read() | Return last angle written |
| attached() | Check if servo is connected |
| detach() | Disconnect servo from pin |

## Basic code

```
#include <Servo.h> Servo myservo; void setup() { myservo.attach(9); myservo.write(90); } void loop() { myservo.write(0); delay(1000); myservo.write(180); delay(1000); }
```

**Memory: Attach → Write → Read → Check → Detach.**

# Final 1-Minute Recall

| Topic | Must remember |
|---|---|
| IoT flow | Sensor → Arduino → Actuator |
| Analog vs digital | Continuous value vs HIGH/LOW |
| MQ-2 | Gas/smoke; gas ↑ means resistance ↓ |
| Obstacle | IR reflection; obstacle present = LOW |
| HC-SR04 | Trigger sends; Echo time gives distance |
| Distance formula | cm = (duration/2)/29.1 |
| LDR | Light ↑ means resistance ↓ |
| Heartbeat | Blood/light pulse; BPM = 60,000 / interval(ms) |
| Actuator | Output device that performs action |
| Relay | OFF: COM–NC; ON: COM–NO |
| Servo | Closed-loop position motor; write(angle) |

## Exam Answer Framework

For any sensor, write in this order: **(1) Definition/purpose** → **(2) Pins** → **(3) Working principle** → **(4) Arduino interface/function** → **(5) Formula or code logic** → **(6) Application.** For actuators, write: **definition** → **type/terminals** → **working** → **Arduino connection/code** → **applications.**

## Most Important Keywords

AnalogRead • DigitalRead • Serial Monitor • IR reflection • potentiometer • Trigger • Echo • 10 µs • pulseIn() • delayMicroseconds() • voltage divider • LDR • capillary blood volume • millis() • BPM • actuator • COM/NC/NO • closed-loop control • Servo library.

## ONE COMPLETE UNIT IN ONE LINE

**The sensor detects a condition, Arduino processes its electrical signal, and an actuator performs the required real-world action.**